

International Cooperative Study Report

Development of EcoQuest: A Gamified Mobile Application for Promoting Eco-Friendly Behavior in Ebetsu City

Mr. Kantinan Rompho

This Report Submitted in International Cooperative Study,
1/2026 Semester, Information Technology and Digital Communication Major,
Department of Information Technology and Digital Communication,
Faculty of Science and Technology,
Rajamangala University of Technology Thanyaburi

International Cooperative Study Report

Development of EcoQuest: A Gamified Mobile Application for Promoting Eco-Friendly Behavior in Ebetsu City

Mr. Kantinan Rompho

This Report Submitted in International Cooperative Study,
1/2026 Semester, Information Technology and Digital Communication Major,
Department of Information Technology and Digital Communication,
Faculty of Science and Technology,
Rajamangala University of Technology Thanyaburi

Acknowledgements

I would like to extend my heartfelt gratitude to Associate Professor Fumitake Taniguchi of Hokkaido Information University for the invaluable teachings and guidance during my cooperative study. Professor Taniguchi's expertise and mentorship have been instrumental in my academic journey, and I am truly appreciative of this support.

I would also like to express my sincere thanks to Assistant Professor Keeratiburt Kanchanasatian, my cooperative study adviser, for providing me with the wonderful opportunity to study at Hokkaido Information University. This encouragement and guidance have been a significant factor in my academic growth and personal development.

Additionally, I would like to convey my deep appreciation to Mr. Odajima Keita for the unwavering support and care throughout my time in Japan. This assistance ensured that my experience was smooth and enjoyable.

I am grateful to these individuals for their contributions to my cooperative study and for making this experience both enriching and memorable. Their support has been invaluable to my academic pursuits and personal growth.

Kantinan Rompho

Abstract

Project Title:	Development of EcoQuest: A Gamified Mobile Application for Promoting Eco-Friendly Behavior in Ebetsu City
Organizer:	Mr. Kantinan Rompho 116610906003-6
Job Adviser:	Assoc. Prof. Fumitake Taniguchi (Hokkaido Information University)
Adviser:	Asst. Prof. Keeratiburt Kanchanasatian
Major:	Information Technology and Digital Communication
Faculty:	Faculty of Science and Technology
Academic Year:	1/2026

This project presents the design and development of EcoQuest, a gamified mobile application that encourages residents of Ebetsu City, Hokkaido, to take small environmental actions in daily life and to keep doing them. Users complete real-life quests in five categories (food waste, recycling, plastic reduction, energy saving and community activities), take a photo as proof, and immediately receive points, experience points (XP) and an estimated amount of CO_2e saved. Posts appear in a community feed where other players can cheer or report photos that do not show the quest, and an administrator reviews only reported posts.

To keep users motivated, the application combines levels, a Daily Streak, a Daily Variety Combo, a weekly Eco Bingo card, three-tier achievement medals, multi-day quests and party events for group activities. A Smart Fridge Tracker helps users record food and expiration dates to reduce food waste, and a city impact card shows the combined CO_2e saved by all players. The system was built with Flutter for the Android client, Node.js and Express for the backend on Render, and MongoDB Atlas as the database.

The results show that all main functions work on a real Android device, automated backend tests passed, and a game balance simulation was used to tune rewards. The verification method was revised twice according to the job adviser's feedback, ending with instant rewards plus community reporting to reduce reviewer workload and keep users motivated. User testing with local residents is suggested as the next step.

List of Contents

Acknowledgements i
Abstract ii
List of Contents iii
List of Tables vi
List of Figures vii
Chapter 1 Introduction 1
1.1 Statement and Significance of the Problems 1
1.2 Objectives 1
1.3 Scopes of the Study 2
1.4 Goals 3
1.5 Project Plan 3
Chapter 2 Literature Review 5
2.1 Gamification and Motivation 5
2.1.1 Definition of gamification 5
2.1.2 Effects of gamification 5
2.1.3 Self-determination theory 5
2.1.4 Habit formation 5
2.2 Environmental Problems in Japan and Ebetsu City 6
2.2.1 Food loss and waste 6
2.2.2 Plastic waste and refill products 6
2.2.3 Recycling and store collection 6
2.3 Estimating the CO_2e Impact of Everyday Actions 6
2.4 Technologies Used in Development 7
2.4.1 Flutter and Dart 7
2.4.2 Node.js and Express 7

2.4.3 MongoDB Atlas and Mongoose 7
2.4.4 Socket.IO 7
2.4.5 Render 7
Chapter 3 Research Methodology 8
3.1 Materials and Equipment 8
3.1.1 Hardware 8
3.1.2 Software 8
3.2 System Design 8
3.2.1 System architecture 8
3.2.2 Database design 9
3.2.3 Quest design and scoring 20
3.2.4 Quest completion and photo proof 21
3.2.5 Community feed, reporting and moderation 23
3.2.6 Progression and motivation systems 24
3.2.7 Party events 25
3.2.8 Smart Fridge Tracker 26
3.2.9 Environmental impact tracking 27
3.2.10 Administration 27
Chapter 4 Results 28
4.1 Account and Home 28
4.2 Exploring and Completing Quests 29
4.3 Multi-day Quests and Progress 31
4.4 Community Feed, Impact and Reporting 32
4.5 Smart Fridge, Eco Bingo and Achievements 34
4.6 System Testing 36
4.7 Game Balance Simulation 37

4.8 Improvements Based on the Job Adviser's Feedback 38
Chapter 5 Conclusion and Suggestion 40
5.1 Conclusion 40
5.2 Problems and Solutions 41
5.3 Suggestion 41
References 42
Appendix I Details of the Code 44
1. Quest Score Calculation 44
2. Daily Variety Combo 44
3. Multi-day Quest Rewards 45
4. Instant Reward for Photo Proof 45
5. Reporting a Post 46
6. Sorting Fridge Items (Flutter) 46

List of Tables

Table 1: Main Features of EcoQuest 3
Table 2: Project Plan (24 August – 20 November 2026) 4
Table 3: Data Dictionary of the users Collection 13
Table 4: Data Dictionary of the quests Collection 14
Table 5: Data Dictionary of the questprogresses Collection 15
Table 6: Data Dictionary of the questsubmissions Collection 16
Table 7: Data Dictionary of the questhistories Collection 17
Table 8: Data Dictionary of the bingocards Collection 17
Table 9: Data Dictionary of the fridgeitems Collection 17
Table 10: Data Dictionary of the achievements Collection 18
Table 11: Data Dictionary of the notifications Collection 18
Table 12: Data Dictionary of the parties Collection 18
Table 13: Data Dictionary of the partymembers Collection 19
Table 14: Data Dictionary of the friendships Collection 19
Table 15: Data Dictionary of the chatmessages Collection 19
Table 16: Data Dictionary of the inventoryitems Collection 20
Table 17: Data Dictionary of the userupgrades Collection 20
Table 18: Quest Scoring 20
Table 19: Active Quests in EcoQuest 21
Table 20: Motivation Systems and Rewards 25
Table 21: Main Test Cases and Results 37
Table 22: Game Balance Simulation by Player Type 38
Table 23: Changes Based on Adviser Feedback 39
Table 24: Problems Found During Development and Solutions 41

List of Figures

Figure 1: EcoQuest System Architecture 9
Figure 2: ER Diagram (1): Quests and Rewards 10
Figure 3: ER Diagram (2): Parties and Social Features 11
Figure 4: ER Diagram (3): User-owned Data 12
Figure 5: Quest Completion Flowchart 23
Figure 6: Post Report and Moderation Flowchart 24
Figure 7: Level Curve of EcoQuest 25
Figure 8: Party Event Flowchart 26
Figure 9: Smart Fridge Tracker Flowchart 27
Figure 10: Login Page 29
Figure 11: Home Page 29
Figure 12: Explore Page 30
Figure 13: Quest Detail Page 30
Figure 14: Proof Photo Sheet 31
Figure 15: Reward Animation 31
Figure 16: Multi-day Check-in Ring 32
Figure 17: Progress Page 32
Figure 18: Community Feed and Ebetsu's Impact 33
Figure 19: Report Post Sheet 33
Figure 20: Reported Posts Page (Admin) 34
Figure 21: Party Room 34
Figure 22: Smart Fridge with Sort Options 35
Figure 23: Eco Bingo Card 35
Figure 24: Achievement Medals 36
Figure 25: Profile Page 36
Figure 26: Game Balance Simulation Results 38

Chapter 1
Introduction

1.1 Statement and Significance of the Problems

Environmental problems such as food waste, plastic waste and greenhouse gas emissions are closely connected to the small choices people make every day. In Japan, the Ministry of the Environment estimated that 4.72 million tonnes of food loss and waste were generated in fiscal year 2022, and about half of this amount (2.36 million tonnes) came from households [1]. The Consumer Affairs Agency also estimated that food loss in Japan leads to about 10.46 million tonnes of CO_2 emissions per year, or about 83 kg per person [3]. Single-use plastic, sorting mistakes and unnecessary electricity use add further impact at the household level.

Ebetsu City in Hokkaido, where Hokkaido Information University is located, faces the same everyday issues: food waste, plastic waste, waste reduction and recycling. Most residents already know that small actions such as finishing a meal, sorting waste correctly or turning off unused lights are good for the environment. However, these actions are easy to forget. People rarely see the result of what they do, and there is little motivation to keep doing it every day.

Gamification, the use of game design elements in non-game contexts [10], is one way to address this motivation gap. Points, levels, streaks, badges and social features can give users immediate feedback and a reason to come back. Research has shown that gamification can have positive effects on engagement and behavior when it is designed for the context and the users [11].

For these reasons, I developed EcoQuest, a mobile application that turns everyday eco-friendly actions into quests. Users complete real-life quests, take a photo as proof, earn rewards immediately, and see how much CO_2e they and the whole city have saved together. The project follows the idea "Start Small ⟶ Build Habits ⟶ Increase Participation ⟶ Create Environmental Impact".

1.2 Objectives

1.2.1 To develop a mobile application that turns everyday environmental actions into quests that users can complete and prove with a photo.

1.2.2 To design game mechanics that motivate users to keep taking part, such as levels, streaks, combos, Eco Bingo, medals and group events.

1.2.3 To show the estimated environmental impact (CO_2e) of each user and of the whole community in a way that is easy to understand.

1.2.4 To apply full-stack development skills, including mobile development, REST API design, database design and cloud deployment.

1.3 Scopes of the Study

1.3.1 The client is an Android application developed with Flutter. All text in the application is in English, and the target users are residents and students in Ebetsu City.

1.3.2 The backend is a Node.js and Express REST API deployed on Render, with MongoDB Atlas as the database and Socket.IO for real-time chat.

1.3.3 The application has 21 active quests in five categories: food waste, recycling, plastic reduction, energy saving and community activities (17 solo quests and 4 party quests).

1.3.4 The main features of the system are listed in Table 1.

Feature	Description
Account	Register and log in with email and password, guest login, password reset by email OTP
Quests	Solo quests, multi-day Food Saver quests (1, 3 and 7 days) and party quests
Photo proof	A photo is required to complete a quest; reusing the same photo is blocked
Rewards	Points, XP and levels, Daily Streak, Daily Variety Combo, Eco Bingo, achievement medals
Community	Community feed with cheers and reports, friends, chat and party rooms
Smart Fridge	Record food items with expiration dates, sort the list and receive reminders
Impact	Personal CO_2e saved and the "Ebetsu's impact" card for the whole city
Admin	Review reported posts, remove photos or take back points, testing tools

Table 1: Main Features of EcoQuest

1.3.5 CO_2e values are estimates based on public Japanese data. Quests whose impact cannot be measured reliably in CO_2e (for example, waste sorting or community cleanup) are not counted in CO_2e. Photos are not checked automatically by image recognition.

1.4 Goals

1.4.1 A working mobile application and backend that allow users to complete environmental quests and receive rewards immediately.

1.4.2 Game mechanics that encourage users to try different quests and to continue over many days.

1.4.3 A community feed and city impact summary that let users see that their combined actions make a real difference.

1.5 Project Plan

The cooperative study took place from 24 August to 20 November 2026 (13 weeks). Table 2 shows the project plan by week.

List of Works	Aug	Sep				Oct					Nov		
	1	2	3	4	5	6	7	8	9	10	11	12	13
1. Orientation and study of environmental issues													
2. Requirement analysis and system design													
3. Flutter app development (UI)													
4. Backend API and database development													
5. Game systems (streak, party, medals)													
6. CO_2e research and game balance													
7. Revisions from adviser feedback													
8. Testing on a real Android device													
9. User testing and final improvements													
10. Poster and report preparation													

Table 2: Project Plan (24 August – 20 November 2026)

Chapter 2
Literature Review

In the development of the project "Development of EcoQuest: A Gamified Mobile Application for Promoting Eco-Friendly Behavior in Ebetsu City", I studied and gathered relevant theories, data and technologies as follows:

2.1 Gamification and Motivation

2.2 Environmental Problems in Japan and Ebetsu City

2.3 Estimating the CO_2e Impact of Everyday Actions

2.4 Technologies Used in Development

2.1 Gamification and Motivation

2.1.1 Definition of gamification

Deterding et al. define gamification as "the use of game design elements in non-game contexts" [10]. Common elements include points, levels, badges, leaderboards, challenges and progress bars. The goal is not to turn an activity into a full game, but to make it more engaging by adding game-like feedback and goals.

2.1.2 Effects of gamification

Hamari, Koivisto and Sarsa reviewed 24 empirical studies and found that gamification generally has positive effects, but the results depend strongly on the context and on the users [11]. This suggests that rewards should be designed carefully: they should support the real activity instead of replacing it, and they should not make users feel controlled or punished.

2.1.3 Self-determination theory

Self-determination theory explains that people are more motivated when three basic needs are met: autonomy, competence and relatedness [12]. In EcoQuest, users can choose which quests to do (autonomy), see their levels and medals grow (competence), and share their actions with others in the feed and in party events (relatedness). This theory also supported the decision to give rewards immediately instead of making users wait for other people to approve their proof.

2.1.4 Habit formation

Lally et al. found that it took a median of 66 days for a new daily behavior to become automatic, with a wide range between people [13]. Because habits take time,

EcoQuest includes features that reward continuing over many days, such as the Daily Streak, multi-day Food Saver quests and a weekly Eco Bingo card.

2.2 Environmental Problems in Japan and Ebetsu City

2.2.1 Food loss and waste

Japan generated 4.72 million tonnes of food loss and waste in fiscal year 2022, of which 2.36 million tonnes came from households [1]. The Ministry of the Environment also reports the amount by type, such as leftovers (食べ残し), food thrown away without being used (直接廃棄) and excessive removal when preparing food (過剰除去) [2]. These types match daily actions that users can change, such as finishing meals, using leftover ingredients and checking expiration dates.

2.2.2 Plastic waste and refill products

According to the Ministry of the Environment's Plastics Smart campaign, refill packs (つめかえパック) can reduce the amount of plastic used by about 70–80% compared with new bottles [5]. Bringing a shopping bag, using reusable containers and choosing refill products are simple actions that reduce single-use plastic.

2.2.3 Recycling and store collection

Many supermarkets in Japan have collection boxes (店頭回収) where customers can return clean food trays, milk cartons, egg packs and PET bottles. Returning these items helps them to be recycled instead of being thrown away. EcoQuest includes a "Return Containers to the Store" quest where users also enter what they returned and how many items, so the total number of returned items can be shown for the whole city.

2.3 Estimating the CO_2e Impact of Everyday Actions

To show users the impact of their actions, each quest was given an estimated CO_2e value where reliable data exists. Japanese data was used whenever possible, for example the food loss statistics above [1][3] and the CO_2 emission factor of Hokkaido Electric Power for electricity-saving quests [4]. Quests whose impact cannot be converted into CO_2e reliably, such as waste sorting and community cleanup, are given no CO_2e value instead of a guessed one.

To avoid overclaiming, each quest counts toward CO_2e at most once per day, even if the user repeats it, and similar quests that overlap share a daily limit. To make the numbers easier to understand, the total is also compared with the amount of CO_2

absorbed by Japanese cedar trees, which the Forestry Agency estimates at about 8.8 kg per tree per year for trees aged 36–40 years [6].

2.4 Technologies Used in Development

2.4.1 Flutter and Dart

Flutter is an open-source UI toolkit by Google for building natively compiled applications from a single codebase using the Dart language [14]. It was used to build the Android client, with the Provider package for state management.

2.4.2 Node.js and Express

Node.js is a JavaScript runtime for building server applications [15], and Express is a minimal web framework for Node.js that makes it easy to build REST APIs [16]. The EcoQuest backend uses Express routes for authentication, quests, submissions, feed, parties, friends, chat and administration.

2.4.3 MongoDB Atlas and Mongoose

MongoDB is a document database that stores data as flexible JSON-like documents, and MongoDB Atlas is its managed cloud service [17]. Mongoose provides schemas and models for MongoDB in Node.js [18]. Users, quests, submissions, fridge items and other data are stored in separate collections.

2.4.4 Socket.IO

Socket.IO enables real-time, bidirectional communication between the client and the server [19]. It is used for the world, party and friend chat in the Community page.

2.4.5 Render

Render is a cloud platform for hosting web services [20]. The backend runs as a web service on Render so that the mobile application can be used anywhere without running a local server.

Chapter 3
Research Methodology

In the undertaking of the project "Development of EcoQuest: A Gamified Mobile Application for Promoting Eco-Friendly Behavior in Ebetsu City", a work plan and a step-by-step design were prepared to achieve the objectives. The details are as follows.

3.1 Materials and Equipment

3.1.1 Hardware

- Laptop computer (Windows 10) for development
- Android smartphone for testing on a real device

3.1.2 Software

- Flutter SDK and Dart, Android SDK and Gradle
- Visual Studio Code
- Node.js, Express, Mongoose and Socket.IO
- MongoDB Atlas (cloud database) and MongoDB Community Server (local testing)
- Render (backend hosting), Git and GitHub (version control)
- Claude Code (AI coding assistant used during development)

3.2 System Design

3.2.1 System architecture

EcoQuest uses a client–server architecture, as shown in Figure 1. The Flutter application communicates with the Express REST API over HTTPS using JSON, and with a Socket.IO server for real-time chat. The backend stores data in MongoDB Atlas through Mongoose and sends password-reset codes by email through Gmail SMTP. All game rules, such as reward calculation, combos, Eco Bingo, streaks and medals, are calculated on the server so that users cannot change them on their phones.

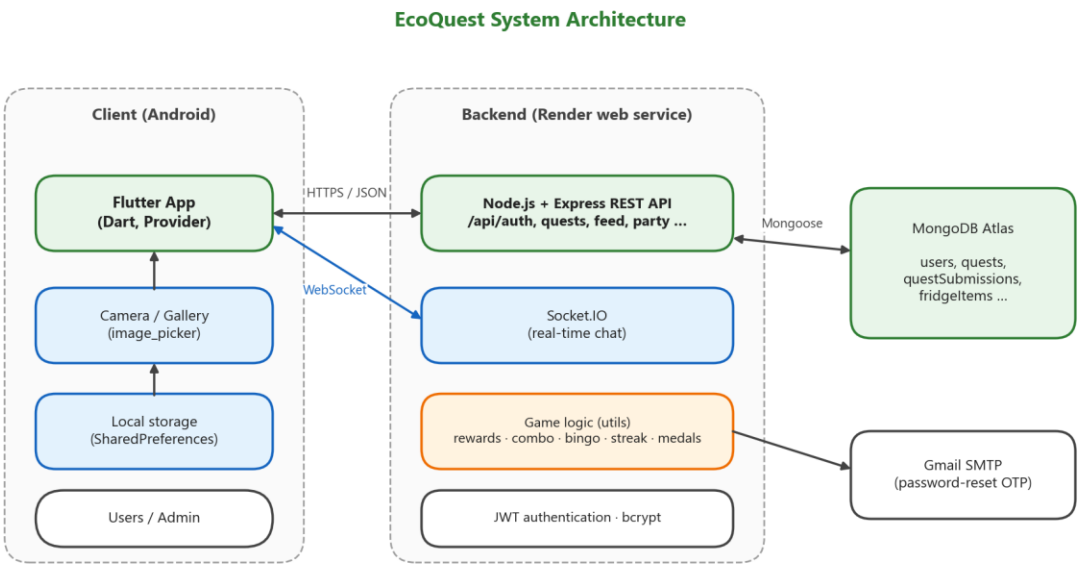


Figure 1: EcoQuest System Architecture

3.2.2 Database design

EcoQuest stores its data in MongoDB Atlas using 15 collections. Instead of putting all data inside one large user document, related data is kept in separate collections and linked with ObjectId references (the ref option in Mongoose). These references work like foreign keys in a relational database, but MongoDB does not check them automatically, so the backend checks them in code. Figures 2 to 4 show the ER diagrams, divided into three groups so that they are easy to read: quests and rewards, parties and social features, and data owned by each user. In the diagrams, PK is the primary key (_id), FK is a field that references another collection, UQ is a unique field, and 1 and N show the cardinality of each relationship. To keep the diagrams readable, they show only the keys and the main fields; every field is listed in the data dictionary.

Some data is embedded inside a document because it is always read together with its parent and is not shared. Examples are the equipped decorations (cosmetics) in users, the proof form (proofForm) in quests, and the reward, awards, reports and form answers (details) in questsubmissions. The channelId field in chatmessages is not a direct reference: it is the text "world", a party _id, or the pairKey of a friendship, depending on channelType, so it is shown with a dashed line.

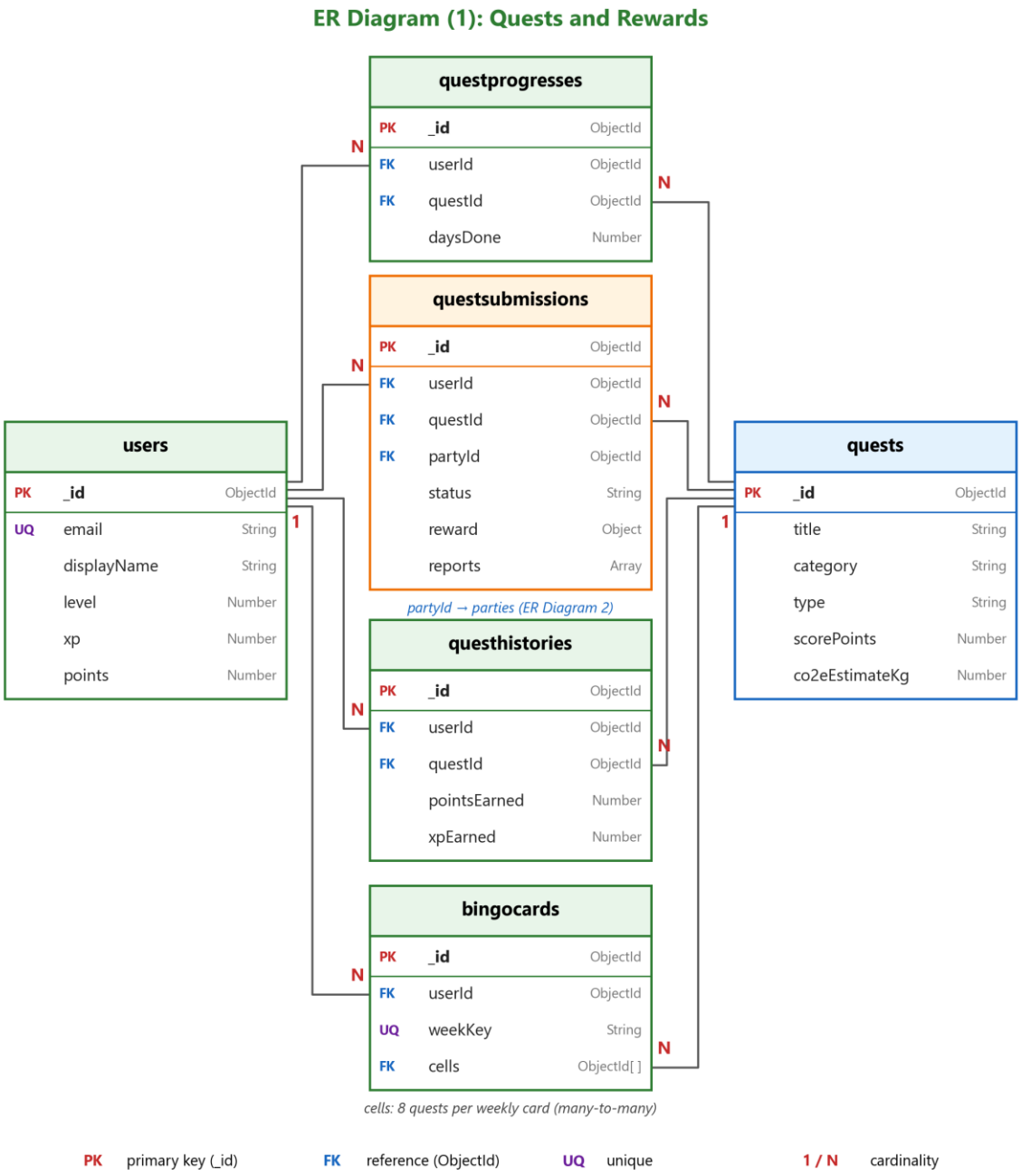


Figure 2: ER Diagram (1): Quests and Rewards

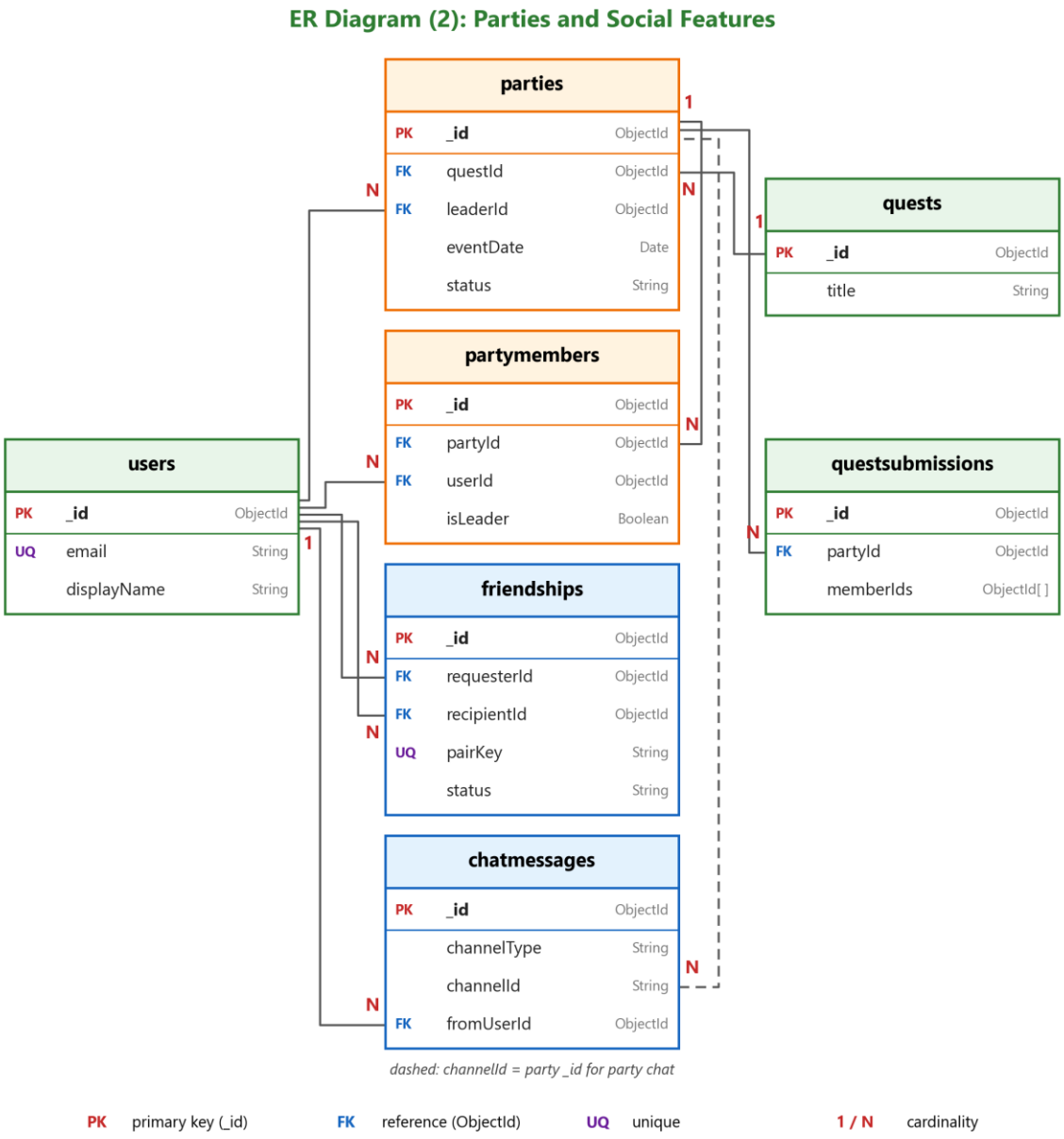


Figure 3: ER Diagram (2): Parties and Social Features

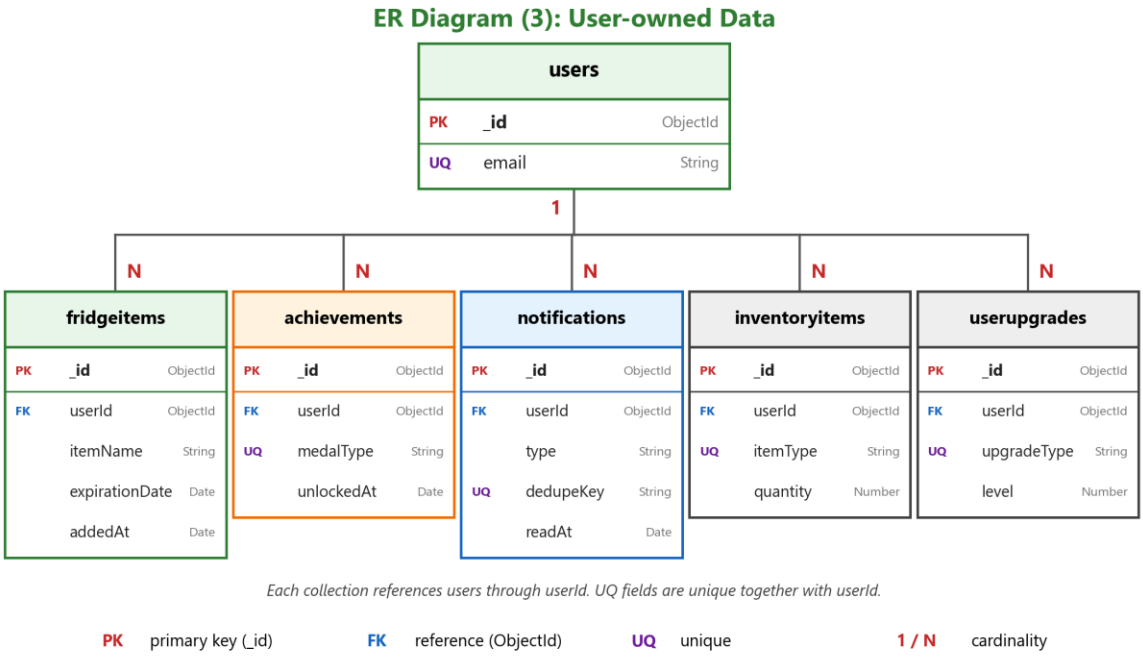


Figure 4: ER Diagram (3): User-owned Data

Tables 3 to 17 show the data dictionary of every collection. All collections also have createdAt and updatedAt fields, which Mongoose adds automatically.

Collection: users — Player accounts, progress and profile settings

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the user
email	String	UQ	Login email (empty for guest accounts)
password	String	–	Password hashed with bcrypt
isGuest	Boolean	–	Whether the account is a guest account
displayName	String	–	Name shown to other players
avatarData	Buffer	–	Profile picture image data
avatarContentType	String	–	Image type of the profile picture (JPEG or PNG)
avatarUpdatedAt	Date	–	Time the profile picture was last changed
notificationsEnabled	Boolean	–	Whether the user receives notifications
level	Number	–	Current level (calculated from xp)
xp	Number	–	Total experience points (never decreases except when points are taken back)
points	Number	–	Points earned from quests and rewards
totalQuestsCompleted	Number	–	Lifetime number of completed quests
streakCount	Number	–	Current Daily Streak in days (1–30)
lastStreakDate	Date	–	Last day counted for the streak
red/blue/greenEnergyExpiresAt	Date	–	Expiry time of boost items (shop feature, currently hidden)
cosmetics	Object	–	Equipped profile decorations: frame, nameStyle, background, effect
resetOtpHash	String	–	Hashed one-time code for password reset
resetOtpExpires	Date	–	Expiry time of the password reset code

Table 3: Data Dictionary of the users Collection

Collection: quests — Quest templates shown in the app

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the quest
title	String	–	Quest name
description	String	–	Short label of the quest type
detail	String	–	Instructions and what the photo should show
imageKey	String	–	File name of the cover image
category	String	–	food_waste, recycling, plastic, energy or community
type	String	–	solo or party
difficulty	String	–	easy, medium or hard
impact	String	–	low, medium or high
scorePoints	Number	–	Points of the quest (difficulty value + impact value)
xpReward	Number	–	XP of the quest
co2eEstimateKg	Number	–	Estimated kgCO_2e saved (null = not counted in CO_2e)
impactCategory	String	–	Type of impact, e.g. "Food Waste Prevented"
impactMetric	String	–	Unit of the impact, e.g. "items"
overlapGroup	String	–	Group of similar quests that share a daily CO_2e limit
minLevelToHost	Number	–	Level required to host a party for this quest
location	String	–	Default event place for party quests
capacity	Number	–	Default maximum number of party members
actionKey	String	–	Special in-app action, e.g. fridge_check
randomPool	String	–	Group of quests from which one is shown each day
poolWeight	Number	–	Chance weight inside the random pool
proofForm	Mixed	–	Extra questions asked with the photo (choices, count, place)
isDaily	Boolean	–	Old once-per-day flag (no longer enforced)
durationDays	Number	–	Number of days for multi-day quests (1 = normal quest)
isActive	Boolean	–	Whether the quest is available
sortOrder	Number	–	Display order in the quest list
alwaysVisible	Boolean	–	Always shown at the top of the Explore page

Table 4: Data Dictionary of the quests Collection

Collection: questprogresses — Quests that a user has started but not finished (unique per user and quest)

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the record
userId	ObjectId	FK	User who started the quest (⟶ users)
questId	ObjectId	FK	Started quest (⟶ quests)
startedAt	Date	–	Time the quest was started
daysDone	Number	–	Days checked in so far (multi-day quests)
lastCheckInDay	String	–	Date of the last check-in (YYYY-MM-DD, Japan time)

Table 5: Data Dictionary of the questprogresses Collection

Collection: questsubmissions — Photo proof of each completed quest, also used as feed posts

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the submission
userId	ObjectId	FK	User who submitted (party leader for party events) (⟶ users)
questId	ObjectId	FK	Completed quest (⟶ quests)
kind	String	–	quest, check_in (one day of a multi-day quest) or party
partyId	ObjectId	FK	Party of a party event (⟶ parties)
memberIds	Array<ObjectId>	–	Party members rewarded for the event
checkIn	Object	–	daysDone, durationDays and finished for multi-day quests
details	Object	–	Form answers: choices, count and place
photoData	Buffer	–	Proof photo (deleted automatically after the day ends)
photoContentType	String	–	Image type of the photo
photoHash	String	–	SHA-256 hash used to block reused photos
status	String	–	approved or rejected (pending only in old data)
approvals, rejections	Array<ObjectId>	–	Votes from the former peer-review system (no longer used)
decidedBy	String	–	instant, or admin when points were taken back
decidedAt	Date	–	Time the submission was approved or rejected
reward	Object	–	Points, XP and combo multiplier given to the submitter
awards	Array<Object>	–	History rows given to each user, used to take back points
reports	Array<Object>	–	Reports from players: userId, reason and time
reportStatus	String	–	open, kept, removed or revoked
hiddenByReports	Boolean	–	Hidden from the feed after three reports
cheers	Array<ObjectId>	–	Users who cheered the post
removedFromFeed	Boolean	–	Whether the post was removed from the feed
removedAt	Date	–	Time the post was removed
removedBy	ObjectId	FK	Owner or admin who removed the post (⟶ users)

Table 6: Data Dictionary of the questsubmissions Collection

Collection: questhistories — Every reward given for a quest; used for CO_2e, medals and Eco Bingo

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the record
userId	ObjectId	FK	Rewarded user (⟶ users)
questId	ObjectId	FK	Completed quest (⟶ quests)
completedAt	Date	–	Time the quest was done
pointsEarned	Number	–	Points received
xpEarned	Number	–	XP received
checkIn	Boolean	–	true = one daily check-in of a multi-day quest

Table 7: Data Dictionary of the questhistories Collection

Collection: bingocards — Weekly Eco Bingo card of each user

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the card
userId	ObjectId	FK	Owner of the card (⟶ users)
weekKey	String	UQ	Week of the card (unique together with userId)
cells	Array<ObjectId>	FK	Nine cells; eight quests and a free center (⟶ quests)
claimedLines	Array<Number>	–	Completed lines that have already been rewarded
fullClaimed	Boolean	–	Whether the full-card bonus has been given

Table 8: Data Dictionary of the bingocards Collection

Collection: fridgeitems — Food items recorded in the Smart Fridge Tracker

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the item
userId	ObjectId	FK	Owner of the item (⟶ users)
itemName	String	–	Name of the food
expirationDate	Date	–	Expiration date and time
quantity	Number	–	Number of items
photoPath	String	–	Old local photo path (legacy data only)
photoData	Buffer	–	Photo of the item
photoContentType	String	–	Image type of the photo
photoRemovedAt	Date	–	Time the photo was deleted after the item expired
addedAt	Date	–	Time the item was added (used for sorting)

Table 9: Data Dictionary of the fridgeitems Collection

Collection: achievements — Medal tiers unlocked by each user

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the record
userId	ObjectId	FK	Owner of the medal (⟶ users)
medalType	String	UQ	Medal and tier, e.g. food_saver:silver (unique together with userId)
unlockedAt	Date	–	Time the medal was unlocked

Table 10: Data Dictionary of the achievements Collection

Collection: notifications — In-app notifications

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the notification
userId	ObjectId	FK	Receiver (⟶ users)
type	String	–	Kind of notification, e.g. quest_complete, achievement, bingo, post_reported
title	String	–	Title shown in the list
message	String	–	Message text
dedupeKey	String	UQ	Key that prevents duplicate notifications (unique together with userId)
data	Mixed	–	Extra data, such as points and XP for the reward animation
readAt	Date	–	Time the notification was read

Table 11: Data Dictionary of the notifications Collection

Collection: parties — Party rooms created from party quests

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the party
questId	ObjectId	FK	Party quest of the event (⟶ quests)
leaderId	ObjectId	FK	User who created and leads the party (⟶ users)
name	String	–	Room name (up to 60 characters)
eventDate	Date	–	Date of the event
location	String	–	Event place
capacity	Number	–	Maximum number of members
status	String	–	open, started or completed (reviewing only in old data)
startedAt	Date	–	Time the leader started the event
completedAt	Date	–	Time the event was completed

Table 12: Data Dictionary of the parties Collection

Collection: partymembers — Members of each party room (unique per party and user)

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the record
partyId	ObjectId	FK	Party room (⟶ parties)
userId	ObjectId	FK	Member (⟶ users)
isLeader	Boolean	–	Whether this member is the leader
joinedAt	Date	–	Time the member joined

Table 13: Data Dictionary of the partymembers Collection

Collection: friendships — Friend requests and friends (one document per pair of users)

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the record
requesterId	ObjectId	FK	User who sent the request (⟶ users)
recipientId	ObjectId	FK	User who received the request (⟶ users)
pairKey	String	UQ	Key made from both user IDs, so each pair has one document
status	String	–	pending or accepted
respondedAt	Date	–	Time the request was accepted

Table 14: Data Dictionary of the friendships Collection

Collection: chatmessages — Messages in world, party and friend chat

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the message
channelType	String	–	world, party or friend
channelId	String	–	"world", a party _id, or a friendship pairKey
fromUserId	ObjectId	FK	Sender (⟶ users)
text	String	–	Message text (up to 1,000 characters)

Table 15: Data Dictionary of the chatmessages Collection

Collection: inventoryitems — Items owned by each user (shop feature, currently hidden)

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the record
userId	ObjectId	FK	Owner (⟶ users)
itemType	String	UQ	Kind of item (unique together with userId)
quantity	Number	–	Number of items
acquiredAt	Date	–	Time the item was received

Table 16: Data Dictionary of the inventoryitems Collection

Collection: userupgrades — Upgrade levels of each user

Field	Type	Key	Description
_id	ObjectId	PK	Unique ID of the record
userId	ObjectId	FK	Owner (⟶ users)
upgradeType	String	UQ	point_booster, xp_booster, party_bonus or quest_unlock (unique together with userId)
level	Number	–	Current upgrade level

Table 17: Data Dictionary of the userupgrades Collection

3.2.3 Quest design and scoring

Each quest has a difficulty and an impact level. The score of a quest is the sum of the two values, as shown in Table 18. For example, a quest with medium difficulty and high impact gives 10 + 15 = 25 points. In the current version, the XP reward is equal to the points.

Level	Difficulty value	Impact value
Easy / Low	5	5
Medium / Medium	10	10
Hard / High	15	15

Table 18: Quest Scoring

The 21 active quests are listed in Table 19. CO_2e values come from the research described in Section 2.3; "–" means the quest is not counted in CO_2e. Multi-day Food Saver quests (marked *) give 12 points per day plus a finish bonus, explained in Section 3.2.4.

Quest	Category	Type	Difficulty / Impact	Points	kgCO_2e
Check Your Food & Expiration Dates	Food waste	Solo	Easy / Low	10	–
Finish Your Meal	Food waste	Solo	Easy / Low	10	0.05
Use Leftover Ingredients	Food waste	Solo	Easy / Low	10	0.05
Food Saver — 1 Day	Food waste	Solo	Easy / Medium	15	0.12
Food Saver — 3 Days	Food waste	Solo (3 days)	Medium / Medium	20*	0.35
Food Saver — 7 Days	Food waste	Solo (7 days)	Hard / Medium	25*	0.81
Sort Waste Correctly	Recycling	Solo	Easy / Low	10	–
Reuse a Plastic Bottle	Recycling	Solo	Easy / Low	10	–
Return Containers to the Store	Recycling	Solo	Easy / Low	10	–
Bring Your Own Shopping Bag	Plastic	Solo	Easy / Low	10	0.01
Buy Refill Products	Plastic	Solo	Easy / Low	10	0.05
Use Refillable Laundry Detergent	Plastic	Solo	Easy / Medium	15	0.07
Use Refillable Dish Soap	Plastic	Solo	Easy / Medium	15	0.04
Use Reusable Food Containers	Plastic	Solo	Easy / Medium	15	0.02
Avoid Single-Use Plastic for One Day	Plastic	Solo	Medium / Low	15	0.12
Turn Off Unused Lights	Energy	Solo	Easy / Low	10	0.06
Unplug Unused Devices	Energy	Solo	Easy / Low	10	0.06
Community Cleanup	Community	Party	Hard / High	30	–
Local Nature Activity	Community	Party	Medium / High	25	–
Neighborhood Recycling Drive	Recycling	Party	Medium / Medium	20	–
Community Garden	Community	Party	Medium / Medium	20	–

Table 19: Active Quests in EcoQuest

3.2.4 Quest completion and photo proof

Figure 5 shows how a user completes a quest. The user starts a quest on the Explore page, does the real activity, and then takes a photo as proof. Some quests also ask for extra information; for example, the "Return Containers to the Store" quest asks what was returned, how many items, and the store name. The server checks that the photo is a valid image and that the same user has not used the exact same photo before

(using a SHA-256 hash). If the check passes, the reward is given immediately and the post appears in the community feed.

The "Check Your Food & Expiration Dates" quest does not need a photo, because the system can check it directly: the user must save at least one new item in the Smart Fridge before completing the quest.

Food Saver quests last 1, 3 or 7 days. The user checks in with a photo once per day. Each check-in gives 12 points and 12 XP, and the last day adds a finish bonus equal to the quest points × (days − 1). As a result, the 3-day quest gives 76 points in total and the 7-day quest gives 234 points, which makes longer quests more attractive than repeating the 1-day quest. If the user misses a day, the count restarts from Day 1.

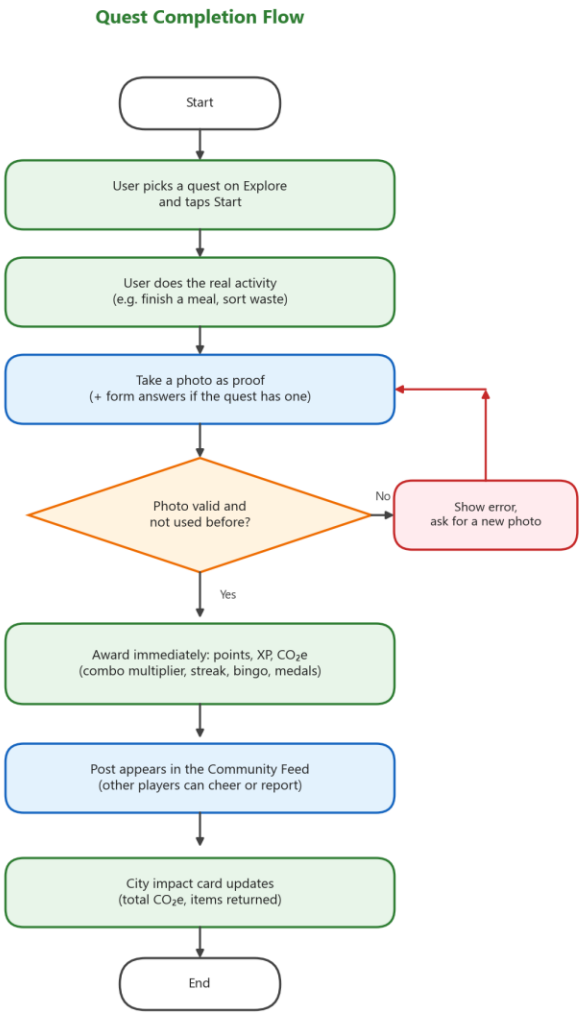


Figure 5: Quest Completion Flowchart

3.2.5 Community feed, reporting and moderation

Every completed quest with a photo appears in the "Today's quests" feed in the Community page, where other players can cheer for it. To save database storage on the free tier, the feed only shows posts from the current day (Japan time), and photos from earlier days are deleted automatically.

The verification method went through three versions. At first, users simply pressed Complete. Following the job adviser's feedback, photo proof was added and two other players had to approve each photo before the user received rewards. Later, the job adviser pointed out that reviewing every action creates a heavy workload, and users who do not

get reviewed lose motivation. The final design, shown in Figure 6, therefore gives rewards immediately and relies on community reporting. Registered players can report a post with a reason (does not show the quest, shows personal information, inappropriate, or other). When three players report the same post, it is hidden from the feed until an administrator checks it. The administrator only reviews reported posts and can keep the post, remove the photo while keeping the points, or remove the photo and take back the points, XP and CO_2e.

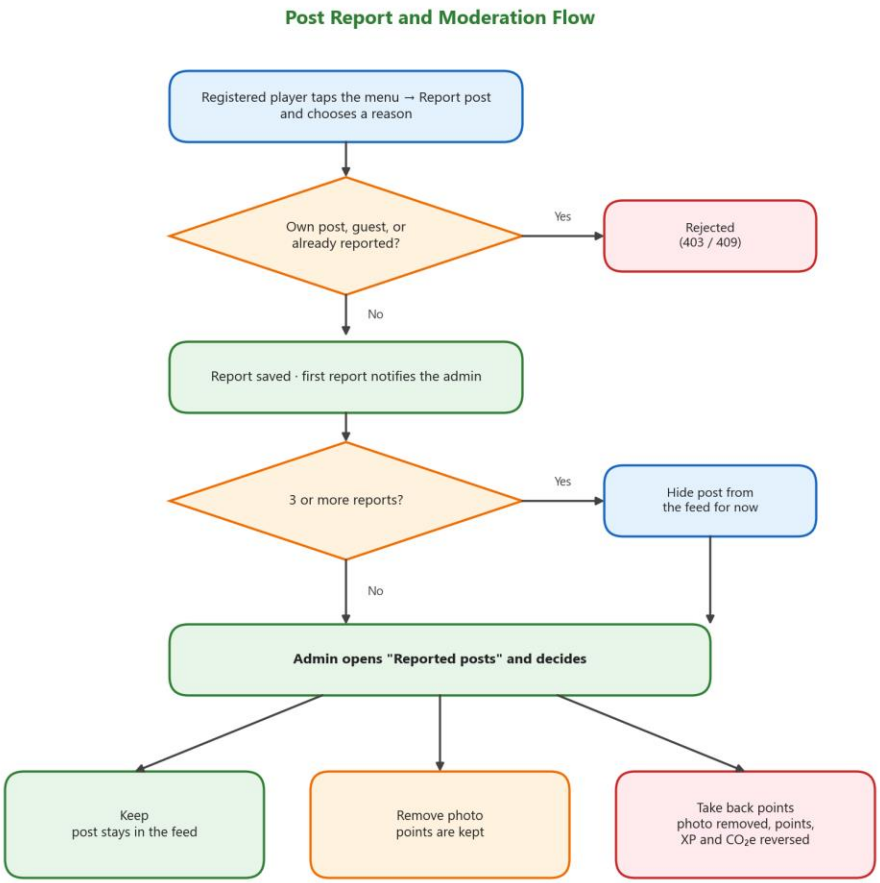


Figure 6: Post Report and Moderation Flowchart

3.2.6 Progression and motivation systems

XP never decreases and determines the user's level. Moving from level L to level L + 1 requires 100 × L XP, so higher levels take longer to reach (Figure 7). Several systems

were added to encourage users to continue and to try different quests instead of repeating one quest. They are summarized in Table 20.

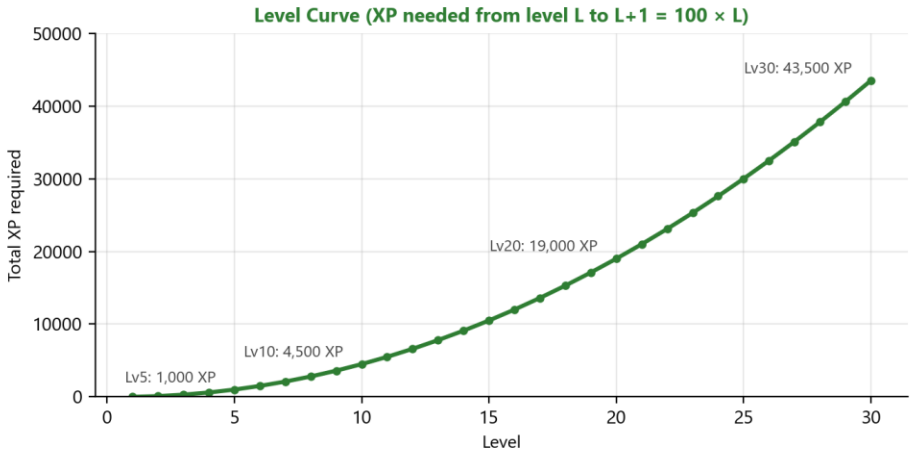


Figure 7: Level Curve of EcoQuest

System	Rule and reward
Daily Streak	Complete at least one quest per day. Rewards on day 7 (+100 P, +50 XP), day 14 (+150 P, +75 XP), day 21 (+200 P, +100 XP) and day 30 (+400 P, +200 XP and an item). Missing a day resets the streak.
Daily Variety Combo	Each different quest on the same day multiplies points and XP by ×1.1, ×1.2 … up to ×1.5. Repeating the same quest gives ×0.75, ×0.5 and at least ×0.25, so repeated good actions still earn something.
Eco Bingo	A weekly 3×3 card with a free center and 8 random quests. Each completed line gives +15 P and +15 XP; a full card gives +50 P and +50 XP more.
Achievement medals	Food Saver, Recycling, Plastic Reduction and Energy Saver medals unlock Bronze, Silver and Gold at 10, 30 and 75 quests. The Community medal unlocks at 1, 5 and 15 party events.
Multi-day quests	12 P and 12 XP for each day checked in, plus a finish bonus on the last day.

Table 20: Motivation Systems and Rewards

3.2.7 Party events

Party quests are activities done in a group, such as a community cleanup or a community garden. As shown in Figure 8, a leader creates a room with the date, location

and capacity, other players join, and the leader starts the event on the event day when there are enough members. After at least 15 minutes, the leader takes one group photo and completes the event. Every member receives the reward at the same time, and members who did not press the button receive a notification with a reward animation.

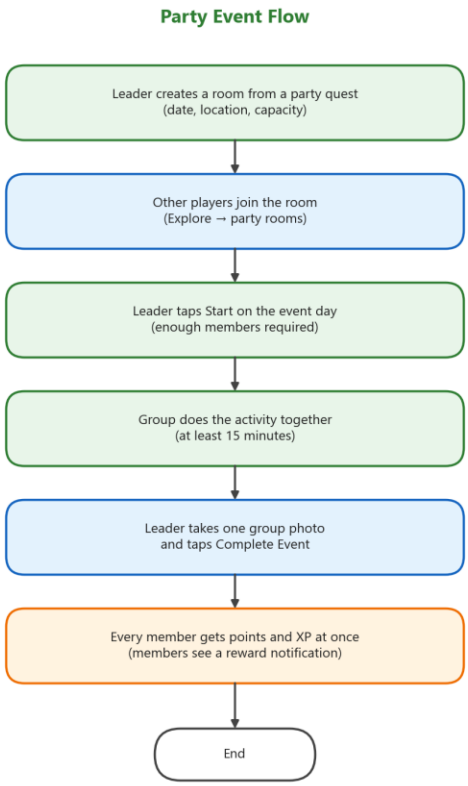


Figure 8: Party Event Flowchart

3.2.8 Smart Fridge Tracker

The Smart Fridge Tracker supports the food waste goal (Figure 9). Users record each food item with its name, expiration date, quantity and an optional photo. The list shows a live countdown and can be sorted by expiration date or by the date the item was added, and the app sends reminders before items expire. When an item expires, its photo is deleted automatically to save storage, but the item stays in the list until the user decides to delete it.

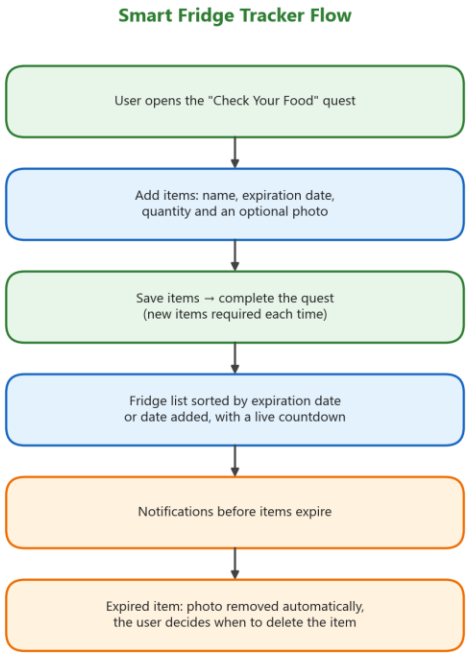


Figure 9: Smart Fridge Tracker Flowchart

3.2.9 Environmental impact tracking

Each user's profile shows the total estimated CO_2e saved, calculated from the quest history with the daily limits described in Section 2.3. The "Ebetsu's impact" card at the top of the Community feed adds up the CO_2e of all players, shows the total for the current week, compares the total with the CO_2 absorbed by cedar trees, and shows the number of containers returned to stores from the quest form answers.

3.2.10 Administration

The administrator account is defined by email address on the server. The administrator can open the "Reported posts" page from the feed banner or the Admin page, and can use testing tools such as unlocking medals. Some features from the early design, such as the item shop, were hidden by feature flags after the job adviser's feedback so that points work as a score rather than a currency.

Chapter 4
Results

Based on the design in Chapter 3, the EcoQuest application and backend were developed and tested on a real Android smartphone with the backend running on Render. The results are as follows.

4.1 Account and Home

Users can register with an email and password, log in as a guest, or reset a forgotten password with a one-time code sent by email (Figure 10). The Home page shows the user's level, XP, points, Daily Streak and quick access to quests (Figure 11).

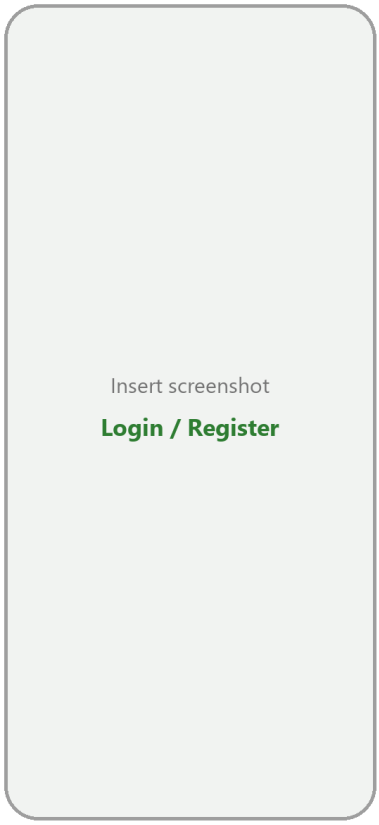


Figure 10: Login Page

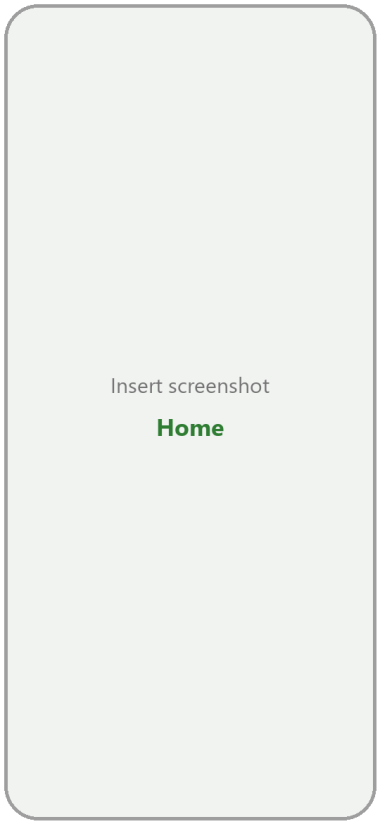


Figure 11: Home Page

4.2 Exploring and Completing Quests

The Explore page lists the available quests with their category, points, combo multiplier and CO_2e (Figure 12). The quest detail page explains what to do and what the photo should show (Figure 13).

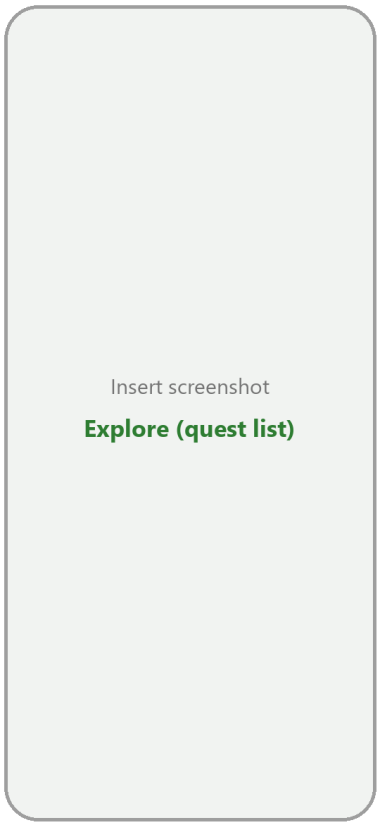


Figure 12: Explore Page

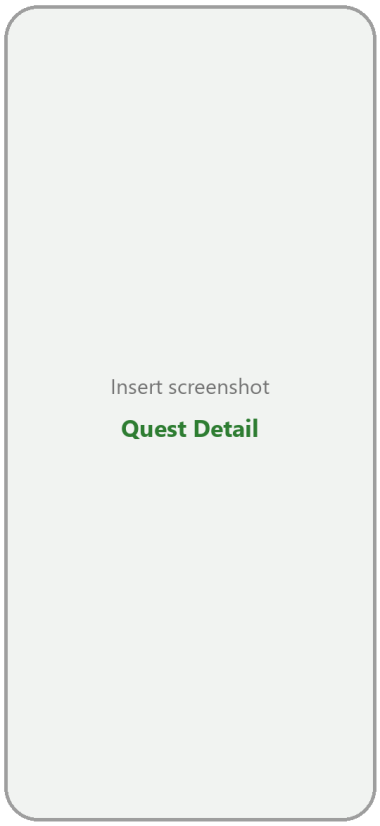


Figure 13: Quest Detail Page

When the user completes a quest, the photo sheet asks for a photo and any extra information, together with a privacy notice (Figure 14). After submitting, the reward animation shows the points and XP earned immediately (Figure 15).

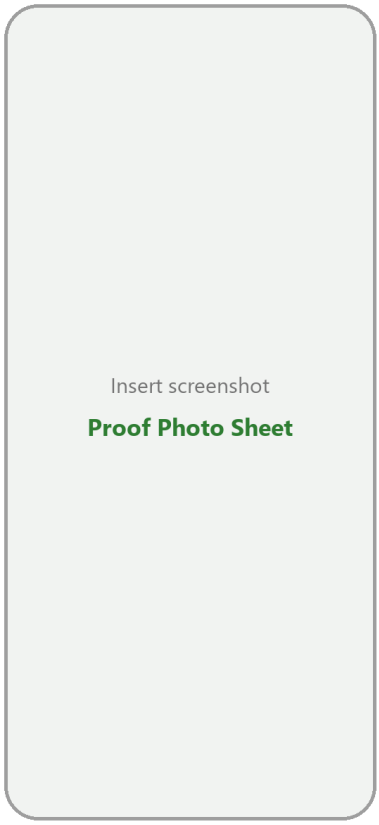


Figure 14: Proof Photo Sheet

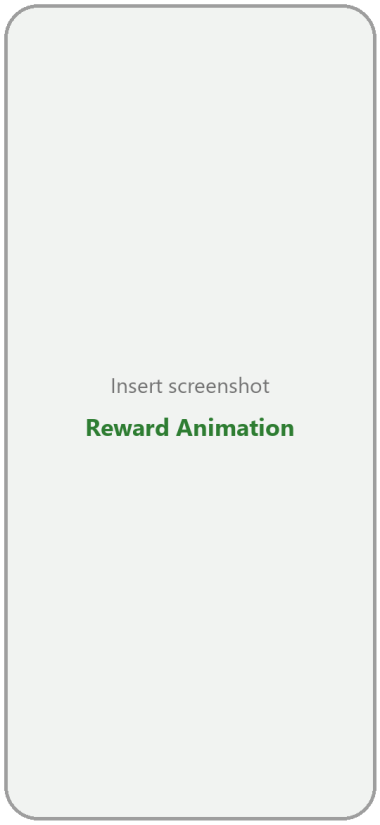


Figure 15: Reward Animation

4.3 Multi-day Quests and Progress

Food Saver check-ins show a ring that fills one segment per day and the points earned for that day (Figure 16). The Progress page lists all quests that the user has started (Figure 17).

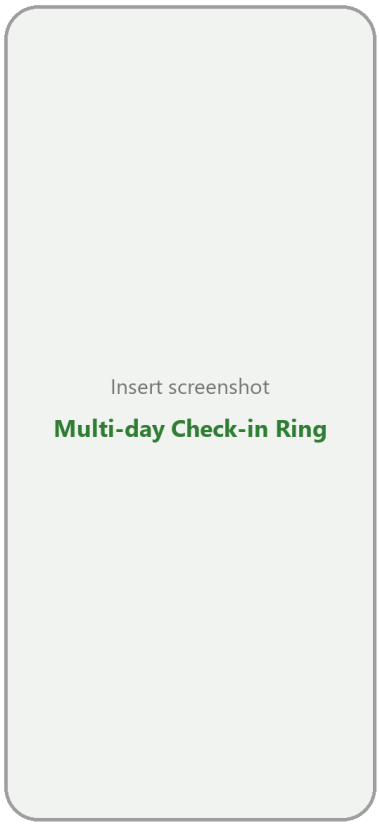


Figure 16: Multi-day Check-in Ring

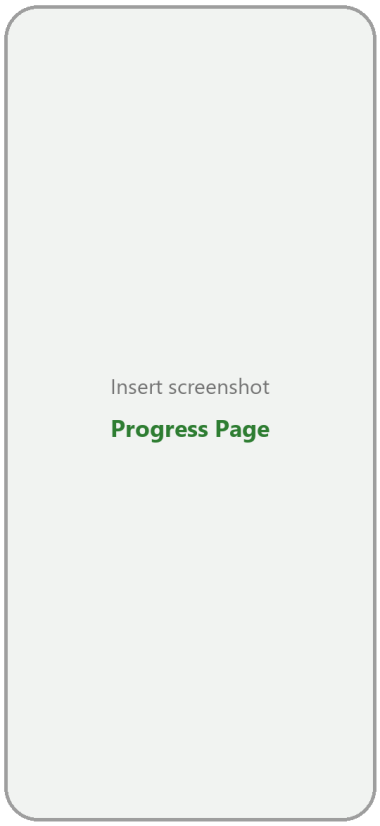


Figure 17: Progress Page

4.4 Community Feed, Impact and Reporting

The Community feed shows the "Ebetsu's impact" card and today's posts from all players, including the quest, photo, CO_2e and cheers (Figure 18). Each post has a menu for reporting with four reasons (Figure 19).

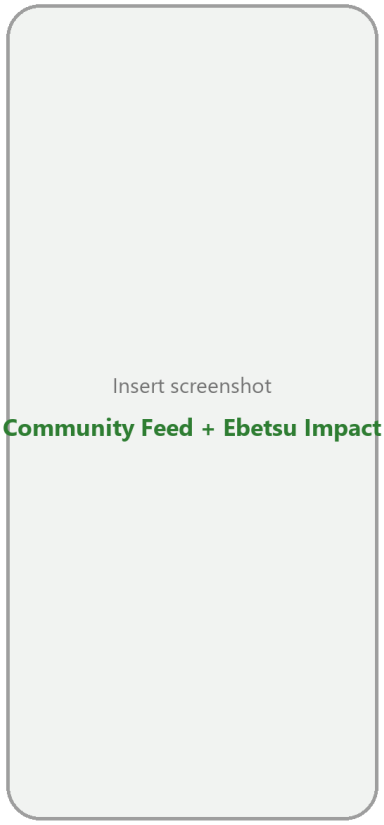


Figure 18: Community Feed and Ebetsu's Impact

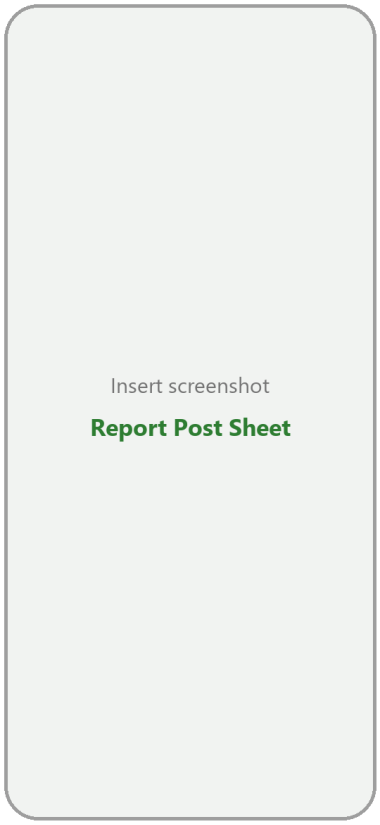


Figure 19: Report Post Sheet

The administrator sees a banner when there are reported posts and reviews them one by one, with the report reasons and the three possible actions (Figure 20). Figure 21 shows a party room where members can see the event details and the leader can start and complete the event.

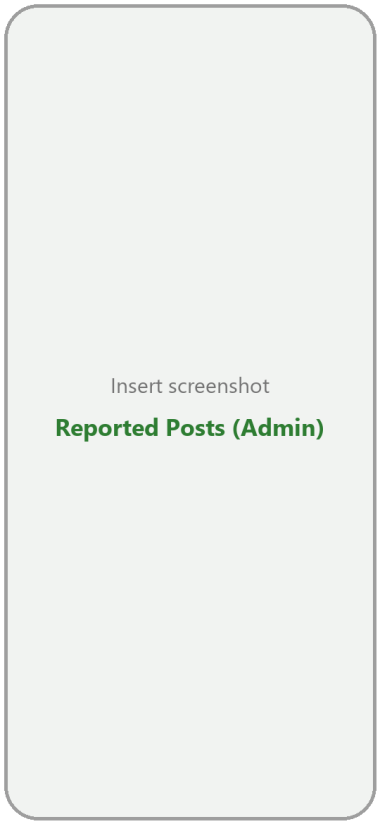


Figure 20: Reported Posts Page (Admin)

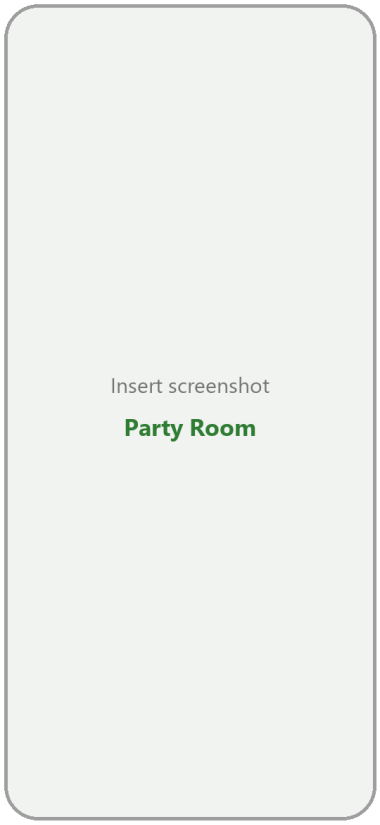


Figure 21: Party Room

4.5 Smart Fridge, Eco Bingo and Achievements

The Smart Fridge page shows the saved items with their expiration countdown and the sort options (Figure 22). Figure 23 shows the weekly Eco Bingo card.

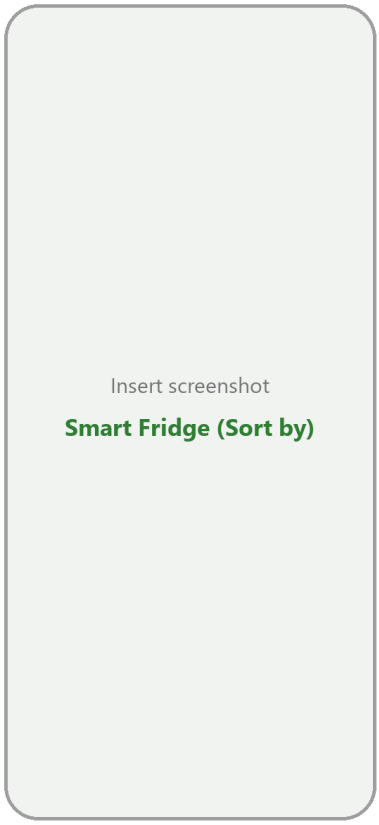


Figure 22: Smart Fridge with Sort Options

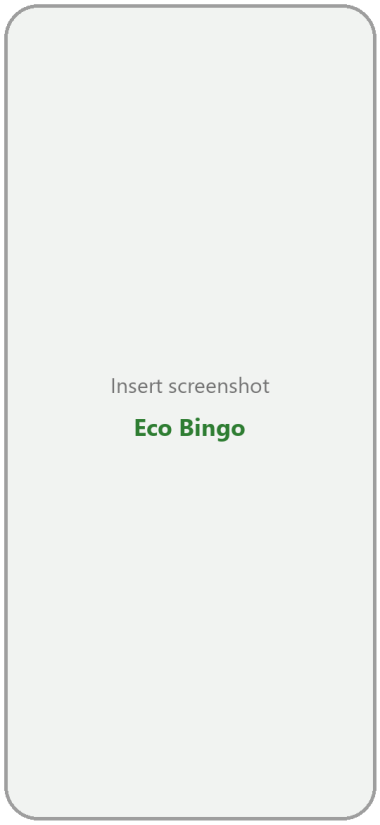


Figure 23: Eco Bingo Card

The achievement page shows each medal category with its Bronze, Silver and Gold tiers and the progress to the next tier (Figure 24). The profile page shows the user's level, total CO_2e saved and other statistics (Figure 25).

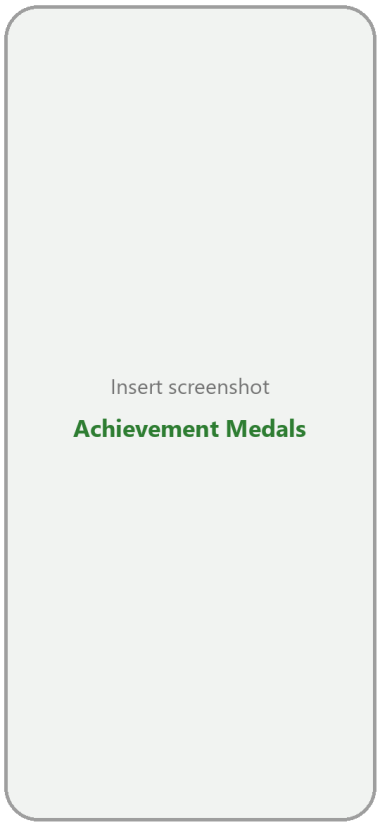


Figure 24: Achievement Medals

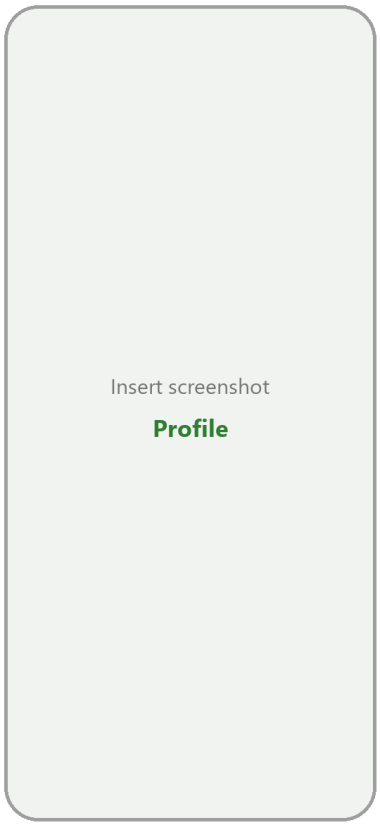


Figure 25: Profile Page

4.6 System Testing

The backend was tested with automated end-to-end scripts that start a temporary local MongoDB database and the API server, create test accounts, and call the API in the same way as the app. The production database was not used for these tests. Table 21 summarizes the main test cases for the final verification and reporting system. All test cases passed. The Flutter code was also checked with the Dart analyzer, and every feature was tested manually on a real Android smartphone.

Test case	Expected result	Result
Submit a quest with a valid photo	Rewarded immediately (201)	Pass
Submit a quest without a photo	Rejected (400)	Pass
Submit the same photo again	Rejected (409)	Pass
Repeat the same quest on the same day	Repeat multiplier applied	Pass
Multi-day check-in (Day 1 of 3)	Daily points given	Pass
Report own post / report twice	Rejected (409)	Pass
Guest reports a post	Rejected (403)	Pass
Three players report the same post	Hidden from the feed	Pass
Admin takes back points	Points, XP and history removed	Pass
Admin keeps a post, new reports arrive	Not reopened	Pass
Leader completes a party event	All members rewarded	Pass
Returned containers total	Correct total, revoked excluded	Pass

Table 21: Main Test Cases and Results

4.7 Game Balance Simulation

To check whether rewards were balanced, a simulation script used the real reward functions from the backend and simulated five types of players for 180 days, 300 times each (Figure 26 and Table 22). The simulation was run on 2 October 2026, before reviewer rewards were removed, so the results include a small amount of reviewer points for the Normal and Active players.

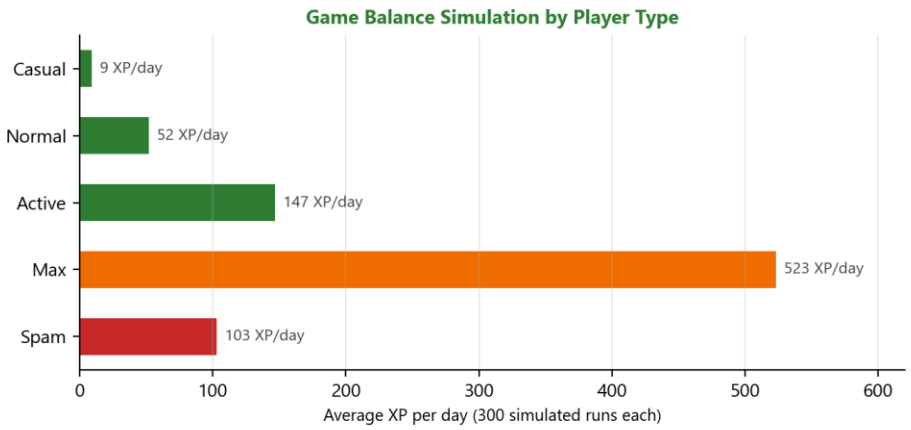


Figure 26: Game Balance Simulation Results

Player type	XP / day	Level after 30 days	Level after 180 days	Days to Lv 10
Casual (3 days/week, 1–2 quests)	9	2.3	5.7	> 180
Normal (most days, 3 quests)	52	5.6	13.7	83
Active (every day, 6 quests)	147	9.1	23.0	31
Max (all quests, repeated)	523	18.0	43.0	9
Spam (one quest, 21 times a day)	103	8.0	19.0	45

Table 22: Game Balance Simulation by Player Type

The simulation found two issues. First, multi-day Food Saver quests gave fewer points per day than the 1-day quest, so they were changed to give daily points plus a finish bonus (Section 3.2.4). Second, repeating one quest many times could earn more XP than doing several different quests. Options to reduce repeated rewards to zero were considered, but the job adviser advised that cutting rewards would discourage environmental actions, so the minimum repeat multiplier of ×0.25 was kept.

4.8 Improvements Based on the Job Adviser's Feedback

The design was improved several times based on feedback from the job adviser. The main changes are shown in Table 23.

Feedback	Change
The item shop is not needed for now	Shop and items hidden by feature flags; points kept as a score
Users should not confirm quests by themselves	Photo proof required for every solo quest and party event
Others should see the activities and feel the real impact	Community feed with cheers and the "Ebetsu's impact" card
Make users want to do the next quest instead of limiting them	Daily quest limit removed; Daily Variety Combo and Eco Bingo added
Cutting rewards for repeats discourages good actions	Repeat multiplier kept at a minimum of ×0.25
Reviewing every action is a heavy workload and users lose motivation	Rewards given immediately; community reporting and admin review of reported posts only
Sort fridge items by date added and expiration date	Sort options added to the Smart Fridge page

Table 23: Changes Based on Adviser Feedback

Chapter 5
Conclusion and Suggestion

From the study and development of EcoQuest, a gamified mobile application for promoting eco-friendly behavior, the following conclusions and suggestions were obtained.

5.1 Conclusion

EcoQuest was successfully developed as an Android application with a cloud backend. Users can complete 21 environmental quests in five categories, prove them with a photo, and receive points, XP and an estimated CO_2e value immediately. Motivation systems such as levels, the Daily Streak, the Daily Variety Combo, Eco Bingo, three-tier medals, multi-day quests and party events give users reasons to continue and to try different actions. The Smart Fridge Tracker supports food waste reduction, and the community feed and city impact card let users see the combined result of everyone's actions.

The objectives of the project were achieved: everyday environmental actions were turned into quests, game mechanics were designed and balanced with a simulation, the environmental impact is shown at both the personal and city level, and full-stack skills were applied from mobile development to cloud deployment. The project also showed the importance of adjusting the design to its real purpose. Because the goal is awareness and participation rather than catching cheaters, the verification method was changed to instant rewards with community reporting.

5.2 Problems and Solutions

Problem	Solution
Free database storage (512 MB) could fill up with photos	Feed shows only today's posts; old proof photos and photos of expired fridge items are deleted automatically; photos are resized before upload
A test photo contained personal information	Privacy notice when taking photos, "Remove post" button, and a report reason for personal information
The free server sleeps and has no scheduler	Background clean-up tasks run lazily when users open common pages, at most once per minute
Pending proofs waited a long time without reviewers	Changed to instant rewards with community reporting
Multi-day quests were less rewarding than 1-day quests	Daily check-in points and a finish bonus

Table 24: Problems Found During Development and Solutions

5.3 Suggestion

Based on the results, I suggest the following improvements for future work:

1. Conduct user testing with residents of Ebetsu City and students to measure whether the application really changes behavior over several weeks, and adjust the rewards based on the results.
2. Add a Japanese language option so that local residents can use the application more easily.
3. Release an iOS version, since Flutter supports both platforms from the same code.
4. Work with Ebetsu City, local supermarkets or environmental groups to create official events and to share the city impact data.
5. Improve the CO_2e estimates with more local data, and consider simple image checks to help administrators find suspicious photos.

References

[1] Ministry of the Environment, Japan. (2024). Estimated Food Loss and Waste Generation in Japan for FY2022. Retrieved from https://www.env.go.jp/en/press/press_02937.html

[2] Ministry of the Environment, Japan. (2024). 食品ロスの発生量の推計値（令和4年度）. Retrieved from https://www.env.go.jp/press/press_03332.html

[3] Consumer Affairs Agency, Japan. (n.d.). 食品ロスによる経済損失及び温室効果ガス排出量の推計結果. Retrieved from https://www.caa.go.jp/notice/entry/047476

[4] Hokkaido Electric Power Co., Inc. (n.d.). 当社のCO2排出係数. Retrieved from https://www.hepco.co.jp/corporate/environment/global_warming/results_co2.html

[5] Ministry of the Environment, Japan. (n.d.). プラスチック・スマート: つめかえパックのプラスチック削減. Retrieved from https://plastics-smart.env.go.jp/plasmaction/kobe/1/

[6] Forestry Agency, Japan. (n.d.). 森林はどのぐらいの量の二酸化炭素を吸収しているの？ Retrieved from https://www.rinya.maff.go.jp/j/sin_riyou/ondanka/20141113_topics2_2.html

[7] NIH Environmental Management System. (n.d.). Life Cycle Environmental Impact of PET Water Bottles. Retrieved from https://nems.nih.gov/Documents/PETWaterBottlesEnvironmentalImpact.pdf

[8] co2everything. (n.d.). CO2e of Plastic Bag. Retrieved from https://www.co2everything.com/co2e-of/plastic-bag

[9] EnergySage. (n.d.). How Many Watts Does a Light Bulb Use? Retrieved from https://www.energysage.com/electricity/house-watts/how-many-watts-does-a-light-bulb-use/

[10] Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining "gamification". Proceedings of the 15th International Academic MindTrek Conference, 9–15.

[11] Hamari, J., Koivisto, J., & Sarsa, H. (2014). Does gamification work? A literature review of empirical studies on gamification. Proceedings of the 47th Hawaii International Conference on System Sciences, 3025–3034.

[12] Ryan, R. M., & Deci, E. L. (2000). Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. American Psychologist, 55(1), 68–78.

[13] Lally, P., van Jaarsveld, C. H. M., Potts, H. W. W., & Wardle, J. (2010). How are habits formed: Modelling habit formation in the real world. European Journal of Social Psychology, 40(6), 998–1009.

[14] Google. (n.d.). Flutter documentation. Retrieved from https://docs.flutter.dev/

[15] OpenJS Foundation. (n.d.). Node.js documentation. Retrieved from https://nodejs.org/docs/

[16] OpenJS Foundation. (n.d.). Express — Node.js web application framework. Retrieved from https://expressjs.com/

[17] MongoDB, Inc. (n.d.). MongoDB Atlas documentation. Retrieved from https://www.mongodb.com/docs/atlas/

[18] Mongoose. (n.d.). Mongoose ODM documentation. Retrieved from https://mongoosejs.com/docs/

[19] Socket.IO. (n.d.). Socket.IO documentation. Retrieved from https://socket.io/docs/

[20] Render. (n.d.). Render documentation. Retrieved from https://render.com/docs

Appendix I
Details of the Code

This appendix shows short excerpts of important code from the EcoQuest project. Comments in the original source code are written in Thai and were removed or shortened here.

1. Quest Score Calculation

File: backend/models/Quest.js

```
const DIFFICULTY_POINTS = { easy: 5, medium: 10, hard: 15 };
const IMPACT_POINTS = { low: 5, medium: 10, high: 15 };


QuestSchema.statics.calculateScore = function (difficulty, impact) {
  return (DIFFICULTY_POINTS[difficulty] || 0) + (IMPACT_POINTS[impact] || 0);
};
```

2. Daily Variety Combo

File: backend/utils/combo.js

```
const COMBO_STEP = 0.1;
const COMBO_MAX = 1.5;
const REPEAT_STEP = 0.25;
const REPEAT_MIN = 0.25;


const round2 = (n) => Math.round(n * 100) / 100;


const comboFor = (prevQuestIds, questId) => {
  const id = String(questId);
  const prev = prevQuestIds.map(String);
  const repeatIndex = prev.filter((q) => q === id).length;
  const distinct = new Set(prev);
  if (repeatIndex > 0) {
    return {
      multiplier: round2(Math.max(REPEAT_MIN, 1 - REPEAT_STEP * repeatIndex)),
      repeat: true,
      distinctToday: distinct.size,
    };
  }
  const distinctIndex = distinct.size + 1;
  return {
    multiplier: round2(Math.min(COMBO_MAX, 1 + COMBO_STEP * (distinctIndex - 1))),
    repeat: false,
    distinctToday: distinctIndex,
  };
};


const nextNewMultiplier = (distinctSoFar) => round2(Math.min(COMBO_MAX, 1 + COMBO_STEP *
distinctSoFar));
```

3. Multi-day Quest Rewards

File: backend/utils/checkInRewards.js

```
const CHECK_IN_DAILY = { points: 12, xp: 12 };


const days = (quest) => Math.max(1, quest.durationDays || 1);


const completionBonus = (quest) => ({
  points: (quest.scorePoints || 0) * (days(quest) - 1),
  xp: (quest.xpReward || 0) * (days(quest) - 1),
});


const finalDayBase = (quest) => {
  const bonus = completionBonus(quest);
  return { scorePoints: CHECK_IN_DAILY.points + bonus.points, xpReward: CHECK_IN_DAILY.xp +
bonus.xp };
};


const dailyBase = () => ({ scorePoints: CHECK_IN_DAILY.points, xpReward: CHECK_IN_DAILY.xp
});
```

4. Instant Reward for Photo Proof

File: backend/routes/quests.js

```
async function submitProof(req, res, quest, photo, kind, checkIn) {
  const now = new Date();
  const submission = await QuestSubmission.create({
    userId: req.userId,
    questId: quest._id,
    kind,
    checkIn: checkIn
      ? { daysDone: checkIn.daysDone, durationDays: checkIn.durationDays, finished:
checkIn.finished }
      : undefined,
    photoData: photo.data,
    photoContentType: photo.contentType,
    photoHash: photo.hash,
    details: req.proofDetails || undefined,
    status: 'approved',
    decidedBy: 'instant',
    decidedAt: now,
  });


  const user = await User.findById(req.userId).select('-avatarData');
  if (!user) return res.status(404).json({ message: 'User not found' });
  const streakMilestone = await applyStreakNow(user);


  const { owner, combo } = await awardSubmission(submission, quest);
  if (!owner) return res.status(404).json({ message: 'User not found' });
  const bingo = owner.bingo || null;
```

5. Reporting a Post

File: backend/routes/feed.js

```
router.post('/:id/report', authMiddleware, async (req, res) => {
  try {
    if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
      return res.status(400).json({ message: 'Invalid post id' });
    }
    const reason = req.body && req.body.reason;
    if (!REPORT_REASONS.includes(reason)) {
      return res.status(400).json({ message: 'Choose a reason' });
    }
    const viewer = await User.findById(req.userId).select('isGuest');
    if (!viewer) return res.status(404).json({ message: 'User not found' });
    if (viewer.isGuest) {
      return res.status(403).json({ message: 'Create an account to report posts' });
    }


    const me = new mongoose.Types.ObjectId(String(req.userId));
    const post = await QuestSubmission.findOneAndUpdate(
      {
        _id: req.params.id,
        status: 'approved',
        removedFromFeed: { $ne: true },
        userId: { $ne: me },
        memberIds: { $ne: me },
        'reports.userId': { $ne: me },
      },
      { $push: { reports: { userId: me, reason, createdAt: new Date() } } },
      { new: true, projection: { photoData: 0 } }
    );
    if (!post) {
```

6. Sorting Fridge Items (Flutter)

File: lib/pages/inventory/fridge_page.dart

```
  List<FridgeItemModel> _sorted(List<FridgeItemModel> items) {
    final list = [...items];
    if (_sort == _FridgeSort.expiration) {
      list.sort((a, b) => a.expirationDate.compareTo(b.expirationDate));
    } else {
      final epoch = DateTime.fromMillisecondsSinceEpoch(0);
      list.sort((a, b) => (b.addedAt ?? epoch).compareTo(a.addedAt ?? epoch));
    }
    return list;
```